

Словник термінів простими словами

Лекції L01–L03, пояснені без зайвого жаргону:
що це таке, навіщо воно і як виглядає на
прикладі бота PayPilot.

L01 Промпт як контракт

L02 Метрики й false confidence

L03 Golden dataset з реальних збоїв

111

термінів у трьох розділах

жовтень 2026

Як читати цей словник

Кожен термін пояснено так, щоб його зрозуміла людина, яка ще не працювала з AI-системами: спочатку суть одним-двома реченнями, потім, де це допомагає, короткий приклад зі стенду PayPilot. Терміни йдуть у порядку лекцій, бо кожне наступне поняття спирається на попереднє. Наприкінці — алфавітний покажчик із номером лекції.

L01

Промпт як контракт

L02

Метрики й false confidence

L03

Golden dataset з реальних збоїв

L01 · Промпт як контракт

Скарга
Дефект
Дефект специфікації
Вимога і побажання
System prompt
Зібраний промпт
Анатомія system prompt
Карта анатомії
Порожній блок
Ефективна специфікація
Zero-shot
Few-shot
Chain-of-thought
ReAct
Тест на верифікованість
Шість типів дефектів специфікації
Суперечність
Неверифікованість
Неповнота
Неоднозначність
Невідстежуваність
Надлишкове обмеження
Доказ дефекту
Розподіл
Severity через бізнес-вплив
Тихий дефект
AI-first-pass
Межі аудиту
Prompt governance
Власник вимоги
Рев'юер зміни
Критерій готовності зміни
Тригери позачергового рев'ю
prompts/CHANGELOG.md
Профіль стенду
Дефект Dnn
PayPilot i Verta
Евал (eval)
Specification Review
Prompt Governance Policy

L02 · Метрики й false confidence

Метрика якості AI-системи
Карта метрик
Три шари
Faithfulness
Answer relevancy
Hallucination rate
Знаменник метрики
Context recall і context precision
Доменна коректність
LLM-as-a-judge
Verbosity bias
Недетермінованість метрики
Вартість прогону
Baseline
Дельта
Рушій правил (engine)
Обчислюваний ground truth
False confidence
Ймовірнісний дефект
Popir (threshold)
Мандат
DeepEval
Promptfoo
Щоденна регресія (daily gate)
Розклад прогонів
Smoke-кейси
False positive метрики
Аудит промпту за 15 хвилин
Гіпотеза правки
Overlay-файли дефектів
Керований годинник
Курований контекст
Quality Bar Proposal
Metrics Map

L03 · Golden dataset з реальних збоїв

Golden dataset
Кейс
Шлях скарги до кейса
JSONL
Оракул (oracle)
Обчислений оракул
Три джерела кейсів
Крайовий випадок
Генерація кейсів із рушіїв
context.clock
tolerance
failure_mode
runs
gate (daily / release)
severity (поле кейса)
Валідація набору
set_hash
Golden (DeepEval)
Assertion
Драбина assertion'ів
Негативна assertion
Статистична assertion
LLM-суддя як assertion
Реєстр assertion'ів ранера
Assertion в інструментах
Ранер (eval runner)
Конвеєр прогону
Природний рівень шуму
Регресія
Перевірка власного датасету
Мок моделі (llm-mock)
Чутливість набору
Кейс, що блимає
Матриця покриття
Залишковий ризик
Рядок аналізу залишкового ризику
Automated Eval Suite

L01

Промпт як контракт

Скарга клієнта — ще не помилка. Помилка з'являється тоді, коли є правило, яке бот порушив. А правила для бота живуть у промпті, тому промпт читають як договір: по розділах, перевіряючи кожен рядок і доводячи поломку кількома прогонами.

Скарга

Коли хтось каже «бот відповів погано». Це лише сигнал, що людині щось не сподобалось, а не доказ помилки. Щоб зрозуміти, чи є помилка, треба знайти правило, яке бот порушив.

Приклад. «Він відповів якось сухо» — скарга є, а правила про «тепло» в промпті немає.

Див. також: Дефект · Дефект специфікації

Дефект

Справжня помилка: бот зробив не те, що написано у вимозі. Немає написаної вимоги — немає й дефекту, є лише чиєсь незадоволення.

Приклад. У промпті «не називай комісію», бот назвав. Це дефект.

Дефект специфікації

Помилка не в боті, а в документі з правилами: щось не дописали, написали суперечливо або так, що перевірити неможливо. Найпідступніший вид, бо формально бот нічого не порушив.

Приклад. Ніде не сказано, що робити, коли даних немає, тому бот вигадує.

Див. також: Шість типів дефектів специфікації

Вимога і побажання

Вимога — правило, яке можна перевірити: зрозуміло, що саме буде порушенням. Побажання — гарні слова, які ніяк не перевіриш.

Приклад. «Відповідай не більше ніж трьома реченнями» — вимога. «Будь корисним», «будь професійним» — побажання.

Див. також: Тест на верифікованість

System prompt

Головна інструкція для бота, яку пише команда. На курсі її читають як договір із клієнтом: кожен рядок — обіцянка, і за її порушення хтось відповідає. Правила в ньому пише бізнес, виконує модель, а наслідки відчуває клієнт.

Приклад. Суд зобов'язав Air Canada виконати те, що її чат-бот пообіцяв пасажиру.

Зібраний промпт	Те, що бот бачить просто зараз: базовий файл плюс усі правки й накладені зверху зміни. Перевіряти треба саме його, а не «оригінал із репозиторію». Див. також: Профіль стенду · Overlay-файли дефектів
Анатомія system prompt	Вісім «розділів», які є в будь-якій інструкції для бота, навіть якщо автор про них не думав: хто бот і як говорить; про що відповідає, а про що ні; кому вірити; коли викликати інструменти; що заборонено; що робити в незвичних ситуаціях; як оформлювати відповідь; приклади. Дев'ятий розділ, якого майже ніколи немає: звідки взялося кожне правило і хто його власник.
Карта анатомії	Таблиця з восьми розділів, де навпроти кожного стоїть позначка: є / слабкий / порожній. Спочатку заповнюють карту, і лише потім читають текст. Так видно не тільки те, що написано, а й те, чого бракує.
Порожній блок	Розділ інструкції, якого немає взагалі. Читаючи текст, його не помітиш, бо нема що читати. Саме такі пропуски дають найнепомітніші помилки. Приклад. Немає інструкції на випадок відсутніх даних, і бот описує умови вкладу, якого в банку не існує. Див. також: Неповнота
Ефективна специфікація	Усе, що впливає на відповідь бота в конкретний момент, а не тільки промпт: ще знайдені документи, відповіді інструментів (бази даних) та історія розмови. Для моделі це один суцільний текст, і промпт у ньому — приблизно чверть. Приклад. Промпт не змінювався, а бот відповів інакше, бо документ із тарифами приїхав розрізаним навпіл.
Zero-shot	Інструкція без прикладів: просто «роби так». Усе тримається на формулюванні, тому будь-яка двозначність одразу шкодить. Приклад. Перевірка: спитати те саме трьома різними словами й порівняти відповіді.
Few-shot	Інструкція з прикладами відповідей. Бот копіює приклади сильніше, ніж слухає правила, тож приклад, що суперечить правилу, мовчки перемагає. Приклад. «Відповідай коротко» плюс два довгих приклади дають довгі відповіді.

Chain-of-thought	Бот розписує хід міркувань крок за кроком. Пастка: міркування виглядає переконливо, але перевіряти треба підсумок, а не пояснення. Приклад. Усі кроки конвертації правильні, а фінальна сума — ні (дефект D25).
ReAct	Бот сам вирішує, коли викликати інструменти: перевірити баланс, подати спір. Опис кожного інструмента теж є інструкцією для бота, хоча його зазвичай ніхто не перевіряє. Приклад. Змініть лише опис інструмента, не чіпаючи промпт. Якщо поведінка змінилась, частина правил живе в коді.
Тест на верифікованість	Три питання до будь-якого правила: на що саме дивимось у відповіді? за яким критерієм вирішуємо «добре чи погано»? який конкретний приклад порушення? Якщо на третє відповіді немає, це не правило, а побажання. Приклад. «Будь емпатичним» провалює два питання з трьох. «Перше речення після повідомлення про втрачену картку визнає ситуацію» проходить усі три.
Шість типів дефектів специфікації	Шість способів, якими ламається текст правил: суперечність, неверифікованість, неповнота, неоднозначність, невідстежуваність, надлишкове обмеження. Це той самий список, яким перевіряють звичайні вимоги, тільки тепер об'єкт — промпт.
Суперечність	Два правила, які не можна виконати одночасно. Людина б перепитала, а бот мовчки обирає одне з них, і щоразу різне. Приклад. «Ніколи не називай комісію» і «покажи ставку у відповіді» (дефект D01).
Неверифікованість	Правило, яке неможливо перевірити, бо незрозуміло, що вважати порушенням. Приклад. «Будь глибоко емпатичним» (дефект D02).
Неповнота	Для якоїсь ситуації просто не написано, що робити, і бот діє на власний розсуд, тобто вигадує. Приклад. Клієнт питає про неіснуючий продукт, бот упевнено описує його умови (дефект D03).

Неоднозначність	Правило можна зрозуміти двома способами. Приклад. «Останні транзакції» — це за тиждень чи за рік?
Невідстежуваність	Невідомо, звідки правило взялося і хто за нього відповідає. Таке правило страшно і прибрати, і залишити. Приклад. Рядок про ліміти без посилання на політику банку.
Надлишкове обмеження	Заборона ширша, ніж потрібно, і через неї не працює нормальний сценарій. Виглядає як обережність, тому її рідко вважають помилкою. Приклад. Заборона називати будь-які комісії, хоча вони опубліковані на сайті банку.
Доказ дефекту	Одна відповідь бота нічого не доводить: наступного разу він може відповісти інакше. Тому те саме питання ставлять щонайменше п'ять разів, записують відповіді дослівно й рахують, скільки разів що сталося. І порівнюють із «чистою» версією без дефектів. Див. також: Розподіл
Розподіл	Підсумок кількох прогонів цифрами. Саме так доводять, що бот нестабільний, замість «мені здалося». Приклад. «3 рази з 5 назвав суму, 2 рази відіслав до тарифів».
Severity через бізнес-вплив	Наскільки помилка серйозна, вирішують за наслідками, а не за технічною причиною. Три питання: скількох клієнтів це зачіпає, що втрачається і чи хтось узагалі помітить. Пишеться людською мовою, щоб зрозумів директор. Приклад. Не «суперечність між рядками 12 і 34», а «кожен другий клієнт не отримує ціну переказу і йде в підтримку».
Тихий дефект	Помилка, яку ніхто не помічає: клієнт задоволений, скарги не буде, а шкода накопичується. Вона дорожча за гучну, бо гучну виправлять після першого тікета, а тиху знайде регулятор або суд. Приклад. Бот увічливо описав умови вкладу, якого немає. Клієнт пішов рахувати платежі за неіснуючими умовами.
AI-first-pass	Перший швидкий прохід, коли документ читає модель і підказує, де можуть бути помилки. Це лише підказки: кожну людина перевіряє на стенді, і у звіт потрапляє тільки підтверджене.

Межі аудиту	Чесний список того, чого перевірка не показала: як часто трапляється помилка, як бот поводить себе там, де інструкції немає, і що приносять документи з бази. Обов'язковий розділ звіту.
Prompt governance	Правила, за якими змінюють промпт, щоб це було так само контрольовано, як зміна коду: у кожного правила є власник, кожен змінює хтось перевіряє, кожна версія має записану причину. Чотири частини: ролі, критерій готовності, що записувати, коли переглядати позачергово.
Власник вимоги	Людина чи відділ, що відповідає за суть правила. До них ідуть із питанням «а чому так?».
	Приклад. Compliance відповідає за заборону розкривати внутрішні пороги, підтримка — за тон.
Рев'юер зміни	Людина, яка перевіряє, як правило записане: чи його можна перевірити і чи не суперечить воно іншим. Не та сама людина, що власник.
Критерій готовності зміни	Чекліст «коли зміну промпту можна випускати»: нове правило перевірюване, не суперечить іншим, названі власник і причина, прогнано п'ять разів до і після, є запис у журналі змін.
Тригери позачергового рев'ю	Події, після яких промпт треба передивитися, навіть якщо його текст не чіпали: змінили модель, оновили базу знань, додали інструмент, стався інцидент.
prompts/CHANGELOG.md	Журнал змін промпту: версія, дата, автор, що змінили, навіщо, якими прогонками підтвердили і що свідомо не вимірювали. Три хвилини зараз економлять години через пів року.
Профіль стенду	Режим навчального стенду PayPilot. <code>clean</code> — усе справно; <code>lesson-NN</code> — увімкнені дефекти конкретного заняття. Можна поставити питання в обох режимах одночасно й порівняти відповіді.
Дефект Dnn	Навмисно вбудована в стенд помилка з номером, щоб студенти могли її знайти.
	Приклад. На L01 активні D01 (суперечність про комісії), D02 (неперевірюваний тон), D03 (немає інструкції для незвичних ситуацій).
	Див. також: Ймовірнісний дефект
PayPilot і Verta	Вигаданий банк Verta і його бот підтримки PayPilot, який відповідає про баланси, комісії, ліміти, конвертацію та спори. Усі приклади й домашні завдання курсу — про них.

Евал (eval)	Автоматична перевірка бота за набором питань і очікуваних відповідей. Головна думка курсу: евал — це виконана специфікація. Без списку правил перевіряти нема що.
Specification Review	Підсумковий документ першої лекції: карта анатомії, таблиця знахідок із доказами, переформульовані правила, межі аудиту.
Prompt Governance Policy	Одна сторінка про те, як у команді змінюють промпт. Має бути настільки простою, щоб команда реально її виконувала вже наступного тижня.

L02

Метрики й false confidence

Скаргу спочатку розкладають на тип помилки, тип помилки вказує на етап, етап підказує метрику. Метрика — це не формула, а звернення до моделі. Число без точки відліку нічого не каже, а зелений дашборд означає лише, що не спрацювали ті помилки, які ви вирішили міряти.

Метрика якості AI-системи

Число, яке міряє один конкретний вид помилки й має зрозумілий діапазон. Не «наскільки відповідь хороша загалом», а, наприклад, «чи не вигадав» або «чи відповів на те питання».

Карта метрик

Таблиця «де ламається > яка це помилка > якою метрикою міряти». Головне правило: не зводити метрики в одне «загальне число», бо воно ховає саме ту помилку, яку шукали.

Приклад. Середнє між faithfulness 0.95 і доменною коректністю 0.55 дає 0.75, за яким не видно, що бот дає неправильні юридичні строки.

Три шари

Три етапи, на яких бот може помилитися. Пошук: не знайшов потрібний документ. Генерація: вигадав або відповів не на те. Дія: знайшов правильне правило, але застосував його неправильно. У кожного етапу свої метрики, і метрика з чужого етапу показуватиме зелене на червоному.

Faithfulness

Чи не вигадав бот нічого поза знайденими документами. Рахується як частка тверджень у відповіді, які підтверджуються контекстом. Важлива межа: якщо знайшовся не той документ, а бот чесно його переказав, faithfulness усе одно висока.

Приклад. Бот назвав комісію 1,5%, якої в документах немає. Faithfulness низька.

Answer relevancy

Чи відповів бот на те питання, яке поставили, а не на сусіднє.

Приклад. Питали про строк спору, отримали правильну розповідь про ліміти. Факти правдиві, relevancy низька.

Hallucination rate

Чи не суперечить відповідь еталонним документам, які ви самі визнали правдою. Схоже на faithfulness, але порівнює з іншим: не з тим, що знайшлося, а з тим, що має бути. Потребує заздалегідь зібраного еталона, тому на повну працює лише з L03.

Знаменник метрики	«Частка чого?» Твердження, документи чи питання? Дві метрики можуть описуватися однаковими словами й міряти різне. Різниця саме в знаменнику. Приклад. «2% галюцинацій» без знаменника нічого не означає: два відсотки тверджень, відповідей чи документів?
Context recall і context precision	Метрики етапу пошуку: чи знайшлося все потрібне і чи не приїхало зайвого. На L02 лише називаються, докладно розбираються на L04.
Доменна коректність	Чи збігається висновок бота з тим, що порахував код банку. Безкоштовна й завжди однакова, бо обходиться без моделі-судді. Головна цінність: ловить помилки, які інші метрики пропускають. Приклад. На справній системі дорівнює рівно 1.00, не «майже». Див. також: Обчислюваний ground truth
LLM-as-a-judge	Спосіб рахувати метрику: іншу мовну модель просять оцінити відповідь за критерієм. Так рахують faithfulness і answer relevancy. Наслідки: кожна оцінка коштує грошей, щоразу трохи різна і має власні упередження.
Verbosity bias	Схильність моделі-судді вважати довшу відповідь кращою, навіть якщо нового в ній нічого. Тому багатослівна відповідь із помилкою може отримати вищий бал, ніж коротка правильна.
Недетермінованість метрики	Та сама відповідь може отримати 0.82 сьогодні і 0.74 завтра, бо суддя теж модель. Різниця в сотих — не новина. Для нестабільних помилок дивляться діапазон із кількох прогонів, а суддю запускають із <code>temperature=0</code>.
Вартість прогону	Скільки звернень до моделі коштує одна перевірка. Рахунок: кейси × (1 + кількість метрик із суддею). Далі множимо на середню довжину звернення в токенах і ціну провайдера, а потім звіряємо з реальним рахунком. Приклад. 50 кейсів і 4 метрики — це 250 звернень до моделі, а не 50.

Baseline	Точка відліку: результат того самого набору на завідомо справній системі. Без неї число нічого не каже. Знімається до прогону з дефектами. Не плутати з «учорашнім продакшеном», який уже може бути зламаним. Приклад. Faithfulness 0.78 — це добре чи погано? Без baseline відповісти неможливо. Див. також: Профіль стенду
Дельта	Різниця між результатом на зламаній системі й baseline. Саме вона є знахідкою. Приклад. «0.71» саме по собі не каже нічого. «Впало на 0.23 відносно справної системи» каже все.
Рушій правил (engine)	Частина коду банку, яка рахує за чіткими правилами й завжди однаково: строки спорів, ліміти, конвертацію. Для питань, що впираються в такі правила, правильна відповідь відома наперед без жодної моделі.
Обчислюваний ground truth	Правильна відповідь, яку дає код, а не людина. Приклад. Рушій каже «спір подати не можна», бот увічливо каже «можна». Ось помилка, і жодна метрика генерації її не бачить (дефект D19).
False confidence	Хибна впевненість: усі метрики зелені, а відповідь шкідлива. Так буває, коли набір метрик не міряє ту властивість, яка насправді ламається. Метрики оцінюють форму відповіді, а наслідок бачить лише еталон. Приклад. Гарно написана відповідь із правильними цитатами з політики і неправильним висновком про строк спору.
Ймовірнісний дефект	Помилка, яка проявляється не щоразу, а приблизно у двох випадках із трьох. Про неї не можна судити з одного прогону: потрібна серія і діапазон. Детермінована помилка проявляється завжди. Приклад. D04, D05 і D25 ймовірнісні; D19 і D20 спрацьовують щоразу.
Поріг (threshold)	Значення метрики, нижче якого реліз зупиняється. Це рішення, а не константа з документації: кожен поріг означає одночасно «скільки помилок зловимо» і «скільки релізів заблокуємо». Занадто високий поріг команда просто вимкне через тиждень.

Мандат	Що зараз найважливіше бізнесу: швидкість виходу (Ship it), жодних проблем із регулятором (Zero regulatory risk), менші витрати (Cut the cost) чи масштабування команди (Scale the team). Саме мандат визначає пороги. Приклад. Під Ship it поріг 0.7 на генерації прийнятний. Під Zero regulatory risk доменна коректність стає 1.0 без винятків.
DeepEval	Інструмент для перевірок на Python, схожий на звичайні тести. Повертає число від 0 до 1 і поріг, тому зручно ставити як «ворота» в CI. Брати, коли поріг уже вирішено.
Promptfoo	Інструмент, де перевірки описують у YAML-файлі й порівнюють варіанти промптів і моделей у таблиці. Брати, коли ще шукаєте формулювання.
Щоденна регресія (daily gate)	Невелика частина перевірок, яка автоматично блокує кожну зміну коду. Туди беруть лише дешеве, стабільне й важливе за мандатом. Решту ганяють рідше. Приклад. Доменна коректність — ідеальний мешканець гейта. Faithfulness на всьому наборі — ні: дорого і блимає.
Розклад прогонів	Три частоти. На кожну зміну: доменна коректність і кілька smoke-кейсів. Щодночі: усі метрики з суддею, трічі. Перед релізом: усе плюс свіжий baseline.
Smoke-кейси	Три-п'ять контрольних питань, по одному на кожен відомий тип помилки. Швидко ловлять грубу поломку, не претендуючи на повну перевірку.
False positive метрики	Зворотна ситуація: метрика червона, а відповідь нормальна. Теж записується, бо часті хибні тривоги привчають команду ігнорувати червоне.
Аудит промπτу за 15 хвилин	Швидкий пошук причини червоної метрики. Метрика каже, який етап зламався; відкриваємо лише цю частину інструкції; шукаємо не хибний рядок, а відсутній; перевіряємо знахідку тестом на три питання; записуємо, що очікуємо від правки. Працює, лише якщо причина в тексті, а не в коді.
Гіпотеза правки	Записане заздалегідь очікування від зміни. Перетворює правку промπτу на експеримент, результат якого можна перевірити. Приклад. «Додаю рядок X. Очікую, що faithfulness підніметься з 0.71 до рівня baseline, а решта не зміниться».

Overlay-файли дефектів	Навчальні дефекти не вписані в базовий промпт, а лежать окремими файлами, що накладаються зверху. Шукати їх у базовому файлі марно.
Керований годинник	Можливість «зупинити час» на стенді. Строки спорів рахуються від дати транзакції, тож без фіксованої дати тест через кілька місяців перестане працювати. Приклад. Перша причина фрази «у мене не відтворюється». Див. також: <code>context.clock</code>
Курований контекст	Набір документів, які ви самі перевірили й визнали еталоном. Потрібен для hallucination gate. Якщо підсунути туди те, що знайшов пошук, метрика перетвориться на faithfulness.
Quality Bar Proposal	Документ для бізнесу, що захищає обрані метрики й пороги: карта метрик, пороги з причинами, компроміс у цифрах, чого набір не ловить, розклад прогонів, розбір одного червоного кейса.
Metrics Map	Перша таблиця Quality Bar Proposal: етап › тип помилки › метрика › знаменник › чому саме вона.

L03

Golden dataset з реальних збоїв

Пам'ять людини — поганий спосіб порівнювати «як було» з «як стало». Тому скаргу перетворюють на кейс, правильну відповідь рахує код, спосіб перевірки обирають із драбини від дешевого до дорогого, і зміну в поведінці бота видно за один прогін.

Golden dataset	Збережений набір перевірених питань із правильними відповідями, який ганяють після кожної зміни бота. Замінює «пам'ятаю, як було минулого тижня». Живе весь курс і стає основою для судді, фреймворку й воріт у CI.
Кейс	Одне питання в наборі разом з усім, що потрібно для перевірки: текст питання, очікувана відповідь, клієнт і дата, хто визначив правильну відповідь, як перевіряти, наскільки серйозна помилка, скільки разів ганяти.
Шлях скарги до кейса	Чотири кроки: чи можна відтворити? записати точний вхід (добрати клієнта й суму з бази); сформулювати, що вважати правильним, і сказати, звідки це відомо; записати кейс. Частина скарг відсіюється, і це нормально. Приклад. Два клієнти сперечаються, 60 чи 120 днів на спір. Рушій показує, що праві обидва: строк залежить від причини. У набір ідуть обидві скарги.
JSONL	Формат файлу: один кейс дорівнює одному рядку. Легко дописувати й легко сказати, у якому рядку помилка.
Оракул (oracle)	Відповідь на питання «звідки ми знаємо правильну відповідь?». Три варіанти: порахував код банку (engine), написано в документі (corpus), вирішила людина (human).
Обчислений оракул	Правильну відповідь не пишуть руками, а беруть викликом коду. Головна фраза лекції: цифри в кейс не пишуть, вони приходять із рушія. Приклад. Суму 2154.35 USD за конвертацію 2000 EUR рахує рушій, а не автор кейса. Бот із дефектом каже 2157.61, і лише звірка з рушієм це ловить.
Три джерела кейсів	Звідки беруться питання: зі скарг (те, що вже помітили), згенеровані кодом (комбінації, які ще не траплялися), крайові випадки (межі правил). Скарги — лише початок, бо найдорожчих помилок ніхто не помічає.

Крайовий випадок	Точка, де правило змінює поведінку. Саме тут усе ламається, і саме цього не буває в скаргах, доки не стане інцидентом. Приклад. Останній день строку і наступний за ним, нульова сума, ліміт, витрачений рівно до копійки.
Генерація кейсів із рушіїв	Скрипт перебирає варіанти, викликає код і сам записує правильні відповіді. Обмеження: питати бота можна лише про клієнтів і транзакції, які є в тестовій базі. Скрипт дає вибір, а не готовий набір: беруть усі межі й по одному прикладу з середини.
context.clock	Зафіксована дата в кейсі. Без неї кейс «на 121-й день спір подати не можна» через рік стане неправильним, бо транзакція постаріє. Кейс без дати — з таймером самознищення.
tolerance	Допустима похибка в сумі, кілька центів на округлення. Не спосіб сховати незрозуміле правило: зavelикий допуск одного дня приховає справжню помилку.
failure_mode	Яку саме помилку ловить кейс, назвою механізму, а не номером навчального дефекту. Дає перевірити, чи набір узагалі щось ловить. Приклад. <code>wrong_spread</code> , а не «D21».
runs	Скільки разів ганяти кейс. Для стабільних перевірок — 1, для нестабільних — 5, і тоді питання не «пройшов?», а «у скількох із п'яти пройшов?». Див. також: Статистична <code>assertion</code>
gate (daily / release)	У який прогін іде кейс: щоденний швидкий чи повний перед релізом. Інакше щоденний прогін ганяє все і стає дорогим. Див. також: Щоденна регресія (<code>daily gate</code>)
severity (поле кейса)	Серйозність помилки з L01: <code>low</code> , <code>medium</code> , <code>high</code> , <code>critical</code> . Заповнюється завжди, бо пізніше саме нею зважують пороги воріт у CI.
Валідація набору	Автоматична перевірка файлу перед прогоном: усі поля на місці, значення з дозволеного списку. Помилка в наборі падає одразу з номером рядка, а не посеред прогону.
set_hash	Короткий «відбиток» файлу набору, який друкується в кожному звіті. Якщо два звіти мають однаковий відбиток і різний результат, змінилася система, а не питання.

Golden (DeepEval)	Назва заготовки кейса в DeepEval: вхід і очікування, а реальна відповідь дописується під час прогону. Курсовий формат трохи інший, тому для перенесення потрібен невеликий «перекладач».
Assertion	Правило, за яким машина вирішує «пройшов чи ні». Оракул каже, яка відповідь правильна; assertion — як перевірити, що бот її дав. Порівнювати тексти дослівно не можна, бо бот щоразу формулює інакше. Приклад. Рушій повертає 2154.35. Бот каже: «після спреду 0.9% ви отримаєте близько \$2,154.35». Треба дістати число й порівняти з допуском.
Драбина assertion'ів	Сім способів перевірки від дешевого до дорогого: точний збіг; число з допуском; потрібне слово або шаблон; заборонене слово; схожість за змістом; збіг у k з n прогонів; оцінка моделлю-суддею. Правило: брати найдешевший спосіб, який ловить помилку.
Негативна assertion	Перевірка «бот НЕ сказав зайвого». У банку це часто важливіше за «сказав потрібне», а пишеться одним рядком. Приклад. У відповіді немає внутрішнього номера транзакції; бот не пропонує подати прострочений спір.
Статистична assertion	Перевірка для нестабільних відповідей: кейс вважається пройденим, якщо збігся, скажімо, у 4 випадках із 5. Жоден інструмент не робить цього «з коробки»: рішення, скільки це «достатньо», лишається за вами.
LLM-суддя як assertion	Найдорожчий спосіб: оцінку виставляє модель. Підходить для тону й пояснень своїми словами. Остання інстанція, а не перша: коштує грошей, нестабільний і має упередження. Його налаштовують на L09.
Реєстр assertion'ів ранера	Назви перевірок у курсовому інструменті: прапорець із відповіді інструмента (<code>tool_result_flag</code>), число з допуском (<code>numeric</code> , <code>tool_grounded_numeric</code>), потрібний підрядок (<code>contains</code>), заборонений шаблон (<code>not_regex</code>), суддя (<code>judge</code>).
Assertion в інструментах	У Promptfoo перевірка — рядок у YAML, а префікс <code>not-</code> робить із будь-якої заборону. У DeepEval перевірка — метрика з балом від 0 до 1 і порогом, яку пишуть на Python. Різниця не в силі, а в тому, що є конфігом, а що кодом.
Ранер (eval runner)	Програма, яка бере кейси, ставить питання боту, застосовує перевірки й пише звіт. Сам по собі набір нічого не перевіряє.

Конвеєр прогону	Набір плюс режим стенду › перевірка файлу › ранер › бот › перевірки › звіт. Режим із дефектами вмикається при запуску, тому той самий набір гаяють і на чистій, і на зламаній системі.
Природний рівень шуму	Скільки кейсів падає на справній системі без жодного дефекту, просто тому, що модель трохи випадкова. Це і показує baseline на <code>clean</code> . Приклад. На демо-наборі з 14 стабільних кейсів baseline рівно 14/14.
Регресія	Погіршення після зміни, видиме порівняно з baseline. Порівняння, що руками займало години, тепер триває один прогін. Приклад. Один доданий рядок «округлюй суми до цілих», і 4 кейси з 14 червоні, бо зникли центи.
Перевірка власного датасету	Якщо на зламаній системі результат такий самий, як на чистій, винен не бот, а набір: у ньому немає кейсів на ці помилки. Перший діагностичний крок.
Мок моделі (llm-mock)	Підмінна «модель», яка відповідає за сценарієм. Дає перевірити, чи правильно бот викликає інструменти і що вони повертають, без реальної моделі й за секунду. Не перевіряє формулювання, бо текст написав сценарій.
Чутливість набору	Чи помічає набір помилку. Додавати треба кейси саме в слабке місце, а не «ще кейсів». Приклад. Шкідлива зміна червонить 5 кейсів із 40 — набір годиться. 1 із 500 — ні.
Кейс, що блимає	Кейс, який то проходить, то ні без змін у системі. Гірший за відсутній, бо привчає не звертати уваги на червоне. Зазвичай причина: взяли дорогу перевірку там, де вистачало числа з допуском.
Матриця покриття	Таблиця «етап × механізм», куди розкладають кейси, щоб побачити порожні клітинки. Порожнє — не помилка, якщо воно назване і зрозуміло, коли закриється.
Залишковий ризик	Чесно записане «чого набір не ловить». Два види: відкладене (знаємо, коли закриємо) і прийняте (вирішили не закривати). Ненормально не знати, що ти його прийняв. Приклад. Відкладене: параметри викликів інструментів, закриємо на L07. Прийняте: інші мови, поки банк працює на одному ринку.

**Рядок аналізу
залишкового ризику**

Три питання до кожної дірки: що не покрито; що буде, якщо воно зламається; чому живемо з цим: відклали чи прийняли. Без третього це список претензій до себе, а не рішення.

Automated Eval Suite

Підсумок третьої лекції: набір кейсів, скрипт генерації й налаштування прогону, які запускаються однією командою й показують різницю між чистою та зламаною системою.

Алфавітний покажчик

Українські терміни за абеткою, латинські — окремою групою. Поруч — лекція, у якій термін визначено.

А		Керований годинник	L02	Рев'юер зміни	L01	Chain-of-thought	L01	
	Анатомія system prompt	L01	Конвеєр прогону	L03	Регресія	L03	Context recall і context precision	L02
	Аудит промпту за 15 хвилин	L02	Крайовий випадок	L03	Реєстр assertion'ів	L03	context.clock	L03
			Критерій готовності зміни	L01	Розклад прогонів	L02	DeepEval	L02
В		Курований контекст	L02	Розподіл	L01	failure_mode	L03	
	Валідація набору	L03	М	Рушій правил (engine)	L02	Faithfulness	L02	
	Вартість прогону	L02		Рядок аналізу залишкового ризику	L03	False confidence	L02	
	Вимога і побажання	L01		С	Few-shot	L01		
Власник вимоги	L01	Скарга		L01	gate (daily / release)	L03		
Г		Метрика якості AI-системи	L02	Статистична assertion	L03	Golden (DeepEval)	L03	
	Генерація кейсів із рушіїв	L03	Мок моделі (llm-mock)	L03	Суперечність	L01	Golden dataset	L03
	Гіпотеза правки	L02	Н	Т	Hallucination rate	L02		
	Д			Надлишкове обмеження	L01	Тест на верифікованість	L01	JSONL
Дельта		L02		Неверифікованість	L01	LLM-суддя як assertion	L03	
Дефект		L01		Невідстежуваність	L01	LLM-as-a-judge	L02	
Дефект специфікації		L01		Негативна assertion	L03	Metrics Map	L02	
Е	Дефект Dnn	L01		Три джерела кейсів	L03	Overlay-файли дефектів	L02	
	Доказ дефекту	L01	Недетермінованість метрики	L02	Три шари	L02	PayPilot і Verta	L01
	Доменна коректність	L02	Неоднозначність	L01	Тригери позачергового рев'ю	L01	Prompt governance	L01
	Драбина assertion'ів	L03	Неповнота	L01	Ч	Prompt Governance Policy	L01	
З		О		Чутливість набору	L03	Promptfoo	L02	
	Залишковий ризик	L03	Обчислений оракул	L03	Ш	prompts/CHANGELOG.md	L03	
	Зібраний промпт	L01	Обчислюваний ground truth	L02	Шість типів дефектів специфікації	L01	Quality Bar Proposal	L02
	Знаменник метрики	L02	Оракул (oracle)	L03	Шлях скарги до кейса	L03	ReAct	L01
Й		П		Щ		runs	L03	
	Ймовірнісний дефект	L02	Перевірка власного датасету	L03	Щоденна регресія (daily gate)	L02	set_hash	L03
	К		Поріг (threshold)	L02	А–З	Severity (поле кейса)	L03	
		Карта анатомії	L01	Порожній блок		L01	Severity через бізнес-вплив	L01
Карта метрик		L02	Природний рівень шуму	L03		Smoke-кейси	L02	
Кейс		L03	Профіль стенду	L01		Specification Review	L01	
	Кейс, що блимає	L03	Р		Assertion в інструментах	L03	System prompt	L01
			Ранер (eval runner)	L03	Automated Eval Suite	L03	tolerance	L03
					Baseline	L02	Verbosity bias	L02
							Zero-shot	L01